

VESTIGE

Field-Service Predictive-Maintenance Data Warehouse & Analytics Platform

Project Documentation

Prepared by Ali Hassan

September 2026

Table of Contents

1. Executive Summary	4
1.1 At a Glance	4
2. Problem Statement & Motivation	5
2.1 Why Two Data Sources, Not One	5
3. Data Sources	6
3.1 AI4I 2020 Predictive Maintenance Dataset (real, public)	6
3.2 Synthetic Field-Service Layer (self-generated, explicitly labeled synthetic)	6
4. Architecture Overview	7
4.1 Why This Shape	7
5. Data Model — Star Schema	8
5.1 Fact Table	8
5.2 Dimension & Bridge Tables	8
5.3 Notable Modeling Decisions	8
6. ETL/ELT Pipeline	10
6.1 Ingestion — src/ingestion/	10
6.2 Generation — src/generators/	10
6.3 Validation — src/validators/	10
6.4 Loading — src/loaders/	10
6.5 Seeding — src/seeds/	10
6.6 Daily Automation — src/pipelines/daily_ingest.py	10
7. Data Quality, Validation & Orchestration	11
7.1 Validation Rules	11
7.2 Why There Is No dq_validation_log Table	11
7.3 Why There Is No Deduplication Check	11
7.4 Orchestration & Scheduling	11
8. Design Decisions & Scope Changes	12
8.1 Warehouse: Azure SQL Database → Aiven PostgreSQL	12
8.2 Orchestration: Azure Data Factory Dropped	12
8.3 dim_date Declined	12
8.4 Fact Table Simplification	12
8.5 dq_validation_log Declined	12
8.6 Public Dashboard Distribution	13
9. BI Dashboard	14
9.1 Layout	14
9.2 Access	14
10. Tech Stack	15
11. Repository Structure	16
12. Functional Requirements Mapping	17
13. Limitations & Non-Goals	18

14. Getting Started	19
14.1 Setup	19
14.2 Create the Schema & Backfill the Warehouse	19
14.3 View the Dashboard	19

1. Executive Summary

Vestige is a small, end-to-end data engineering system that ingests raw industrial sensor/failure data, combines it with a synthetic field-service operational layer, validates and structures the result into a cloud-hosted star-schema data warehouse, and exposes it through a live, DirectQuery-connected Power BI dashboard.

The project covers the full range of core data engineering work: building and maintaining ETL/ELT pipelines, writing SQL for a dimensional warehouse, designing and documenting data models, enforcing data quality and validation, and orchestrating and monitoring scheduled jobs. Its scope was deliberately scaled to stay fully debuggable and defensible end to end — every decision in this document, including the ones that were tried and reversed, can be reasoned about and explained in full, rather than glossed over at a scale that would make that impossible.

The build was not friction-free, and that friction is treated as part of the deliverable rather than something to hide. The original plan targeted Azure SQL Database and Azure Data Factory; both were dropped mid-build for a concrete, external reason (Azure's card-verification requirement) and replaced with Aiven PostgreSQL and GitHub Actions without any loss to the underlying skills being demonstrated. Every such decision — including the ones that were tried and reversed — is recorded in this document with its reasoning, not just its outcome.

1.1 At a Glance

Aspect	Detail
Data sources	AI4I 2020 Predictive Maintenance Dataset (real, public) + a synthetic field-service layer (assets, sites, technicians)
Warehouse	PostgreSQL, hosted on Aiven (free tier)
Orchestration	GitHub Actions — scheduled daily ingest pipeline
Dashboard	Power BI Desktop, connected live via DirectQuery
Language	Python 3.13
Live scale (at time of writing)	335 maintenance events · 206 assets · 120 sites · 23 technicians · 363 failure-mode occurrences

2. Problem Statement & Motivation

Nobody hands a data engineer clean data. Real data engineering work consists of scattered raw data arriving from different places — a sensor feed here, an operational record system there — with the job being to combine, clean, and structure that data so an analyst, a manager, or an operations team can use it without ever having to be technical themselves. Vestige is a deliberately small, honest simulation of exactly that discipline.

Concretely, Vestige is built to answer the kind of operational questions a field-service manager would actually ask:

- Which sites have the most equipment breakdowns?
- Which failure mode happens most often — heat, wear, power, overstrain, or random?
- How is workload distributed across technicians?
- Are breakdowns trending up or down over time?

2.1 Why Two Data Sources, Not One

The AI4I 2020 dataset on its own only says what broke and why — a machine failed, cause: overheating. That is a generic "predict machine failure" ML dataset: useful, but not a field-service project, and this exact genre is heavily overdone as a Kaggle/GitHub portfolio piece.

The synthetic work-order, technician, and site layer is what turns it into a field-service project — it adds the operational context (who fixed it, where, and when) that turns raw sensor readings into a realistic simulation of how field-service operations actually run: equipment breaking down, technicians getting dispatched, and sites accumulating a maintenance history over time.

Combining both sources is the entire point of the project: it demonstrates connecting disparate, real-world-shaped data into one coherent, queryable model — rather than a single-notebook science project.

3. Data Sources

3.1 AI4I 2020 Predictive Maintenance Dataset (real, public)

Source: UCI Machine Learning Repository / Kaggle, "AI4I 2020 Predictive Maintenance Dataset." The dataset contains 10,000 simulated operating instances of one industrial milling-machine type — synthetic sensor data built to closely resemble real sensor behavior, since genuine factory telemetry is generally kept private by the companies that generate it.

Each record carries air temperature, process temperature, rotational speed, torque, and tool wear, together with a binary failure label and an indicator for which of five independent failure modes caused the failure (or none). Of the 10,000 records, 339 represent an actual failure.

- TWF — Tool Wear Failure
- HDF — Heat Dissipation Failure
- PWF — Power Failure
- OSF — Overstrain Failure
- RNF — Random Failure

These 339 real failure records seed the historical portion of the fact table. Because AI4I is a static, non-renewing dataset, ongoing daily records are not drawn from the same finite source file — they are synthetically generated in the same statistical shape, preserving the dataset's documented temperature and power/torque relationships, so the live warehouse keeps growing without ever re-using or exhausting the original data.

3.2 Synthetic Field-Service Layer (self-generated, explicitly labeled synthetic)

No public dataset of real HVAC, plumbing, or electrical company work orders exists, so this layer is generated with a small set of Python scripts using Faker and weighted randomization, and is clearly documented as synthetic throughout the project rather than presented as if it were real operational data.

- Technicians — synthetic name and specialty (HVAC, Electrical, Vertical Transportation, or Hoisting), assigned using demand-based weights, with a randomly generated hire date.
- Sites — synthetic Lahore-based service locations, each with a generated site name, address, and area, using varied residential/business naming templates.
- Assets — synthetic field-service equipment, each assigned one mandatory category and, with progressively lower probability, additional categories, plus a generated install date.
- Maintenance events (fact rows) — each generated event ties a sampled AI4I failure record to a synthetic asset, tying the two data sources together into one coherent operational record.

4. Architecture Overview

Vestige's architecture is intentionally linear and small: a scheduled batch job writes into a cloud-hosted relational warehouse, and a BI tool reads from that same warehouse live. There is no intermediate cache, message queue, or data lake layer — at this project's scale, that would be complexity without a corresponding benefit.

GitHub Actions → Aiven PostgreSQL →(DirectQuery)→ Power BI → Published Dashboard

A scheduled GitHub Actions workflow generates a small daily batch of synthetic maintenance events (1–5 rows) and loads them into a PostgreSQL database hosted on Aiven. Power BI connects to that database live via DirectQuery, so every chart and card reflects the current state of the warehouse rather than a static export taken at build time.

4.1 Why This Shape

- A single warehouse as the source of truth keeps the "combine disparate sources into one coherent model" story honest — both AI4I-derived and synthetic data land in the same star schema, not in separate silos joined only inside the BI layer.
- DirectQuery over a scheduled import was chosen deliberately, specifically so the dashboard is a live reflection of the daily pipeline's output rather than a snapshot that would go stale between manual refreshes.
- GitHub Actions was chosen as the single orchestration tool (see §7) rather than splitting scheduling across two systems, once Azure Data Factory was dropped from scope.

5. Data Model — Star Schema

The warehouse is a star schema centered on `fact_maintenance_event`, where each row represents a maintenance event performed after a failure — not the failure occurrence itself. Two of the three relationships out of the fact table are many-to-many by design and are modeled with bridge tables rather than as foreign keys directly on the fact table.

5.1 Fact Table

Column	Type	Notes
<code>event_id</code>	INT, PK	Surrogate key
<code>asset_id</code>	INT, FK → <code>dim_asset</code>	One asset per event
<code>site_id</code>	INT, FK → <code>dim_site</code>	One site per event
<code>failure_date</code>	DATE	Event date (replaces a separate <code>date_id</code> / <code>dim_date</code> design — see §8.3)
<code>air_temperature</code>	DECIMAL	From AI4I
<code>process_temperature</code>	DECIMAL	From AI4I
<code>rotational_speed</code>	DECIMAL	From AI4I
<code>torque</code>	DECIMAL	From AI4I
<code>tool_wear</code>	DECIMAL	From AI4I

5.2 Dimension & Bridge Tables

Table	Type	Purpose
<code>dim_asset</code>	Dimension	Synthetic field-service assets — install date, linked to one or more categories
<code>dim_asset_category</code>	Dimension	HVAC, ELECTRICAL, VERTICAL_TRANSPORTATION, HOISTING — stored once, reused
<code>dim_site</code>	Dimension	Synthetic Lahore-based service sites
<code>dim_technician</code>	Dimension	Synthetic technicians, each with a category specialty
<code>dim_failure_mode</code>	Dimension	The five real AI4I failure modes only — "no failure" is not modeled as a failure mode
<code>asset_category_assignment</code>	Bridge	Many-to-many: an asset can span more than one category
<code>maintenance_event_technician</code>	Bridge	Many-to-many: an event can involve more than one technician, and a technician can work many events
<code>maintenance_event_failure_mode</code>	Bridge	Many-to-many: an event can involve more than one AI4I failure-mode indicator, since the source indicators are independent of one another

5.3 Notable Modeling Decisions

- No `dim_date` table. Date parts (day, month, year, weekday) are derived directly in PostgreSQL/Python from `failure_date` at query time rather than stored redundantly in a separate dimension — unnecessary overhead at this project's scale.
- Derived fields, not stored fields. `last_failure_date` and `maintenance_count` are computed at query time from the fact table rather than duplicated onto `dim_asset`, avoiding a second source of truth that could drift out of sync.
- No dedicated data-quality log table. A `dq_validation_log` design was considered and rejected in favor of validating at the pipeline layer plus PostgreSQL constraints — see §7.2 for the full reasoning.
- `dim_asset_category` modeled properly as many-to-many. The design initially attached `asset_category` as a one-to-many child table on `dim_asset`; this produced category-name redundancy and was revised to a proper reusable dimension plus bridge-table design.
- `dim_asset` kept minimal. Fields considered and declined include Product ID/Type, description, manufacturer, model, and a separate operational condition status — AI4I does not supply any of these, and inventing them would add unsupported synthetic detail beyond what the source data can justify.

6. ETL/ELT Pipeline

The pipeline is organized into five stages, each isolated in its own Python package, plus a sixth stage that runs the first four on a schedule.

6.1 Ingestion — `src/ingestion/`

- `inspect_ai4i.py` profiles the raw AI4I source file without modifying it: missing values, duplicate source rows, failure records, failure count, and a small row sample for a sanity check.
- `extract_ai4i.py` filters the raw file down to the 339 real failure records and writes `data/extracted/ai4i_failures.csv`, leaving the raw file untouched.

6.2 Generation — `src/generators/`

Produces synthetic assets, sites, technicians, asset categories, and AI4I-style failure records using Faker and weighted randomization, while preserving the source dataset's documented sensor relationships and failure-mode logic.

6.3 Validation — `src/validators/`

Every generated or extracted record is checked before it becomes eligible to load: required fields present, correct types, valid category and failure-mode values, non-future dates, finite non-negative sensor readings, and no duplicate categories on a single asset. Each validator returns the full list of errors found on a record rather than stopping at the first one, so a rejected record can be diagnosed in one pass.

6.4 Loading — `src/loaders/`

Parameterized PostgreSQL inserts per table, including the three bridge-table relationships. Dimension tables are populated before the fact table, since referential integrity requires parent rows to exist first.

6.5 Seeding — `src/seeds/`

Orchestrates `generate` → `validate` → `load` per dimension and per fact, used for the one-time historical backfill that populated the warehouse from a standing start.

6.6 Daily Automation — `src/pipelines/daily_ingest.py`

A scheduled GitHub Actions workflow generates and loads a small daily batch of new maintenance events (1–5 rows), enforcing a hard 10,000-row scope limit on the fact table appropriate to the project's intentionally small scale. Beyond that limit, the current row-oriented loading approach would need to become a bulk-loading, high-scale pipeline — a deliberate line drawn to keep the project's scope honest rather than open-ended.

7. Data Quality, Validation & Orchestration

7.1 Validation Rules

- Numeric sensor fields (torque, rotational speed, tool wear, temperatures) must be non-negative, finite, and within a physically plausible range.
- Every generated record must supply all required fields — no silent nulls.
- Asset categories and failure modes must be drawn from the fixed, defined set of allowed values; asset categories must not be duplicated on a single record.
- Dates (install date, hire date, failure date) must be valid and non-future.

7.2 Why There Is No `dq_validation_log` Table

A dedicated `dq_validation_log` table (`run_id`, `run_timestamp`, `check_name`, `rows_checked`, `rows_flagged`) was part of the original design and was deliberately dropped. The proposed design would only have recorded aggregate counts of flagged rows, not which specific records were affected or why — and GitHub Actions already retains full execution logs for every pipeline run. A second, less detailed log inside the database would have added a maintenance burden without adding practical value over what the orchestration layer already provides for free. Data-quality enforcement instead lives in two places: the pipeline's own validators (§6.3) and PostgreSQL's own constraints (foreign keys, NOT NULL, type constraints) at the schema level.

7.3 Why There Is No Deduplication Check

Deduplication was considered and declined as a data-quality check. Duplicate-looking maintenance records are not inherently invalid in Vestige's data model — a site can plausibly have two maintenance events on the same day — and no valid business key has been defined that would reliably distinguish a true accidental duplicate from two legitimate, separate events.

7.4 Orchestration & Scheduling

GitHub Actions runs the daily `ingest` → `validate` → `load` pipeline on a schedule, with the database connection secrets configured in the repository settings. A recovery/backfill pipeline — originally planned as a second, weekly-scheduled job to reprocess any missed daily run — was explicitly scoped out: the daily batches are small (1–5 rows) and the fact table is capped at 10,000 rows, so an occasional missed run is insignificant at this scale, and GitHub Actions already retains execution history with manual rerun support built in.

8. Design Decisions & Scope Changes

This section is the formal write-up of every material decision made during the build, including the ones that were tried and reversed. Nothing here is smoothed over — a project that only shows the final, clean version of itself is a worse demonstration of real engineering judgment than one that shows the corrections along the way.

8.1 Warehouse: Azure SQL Database → Aiven PostgreSQL

The original plan targeted Azure SQL Database as the warehouse. During environment setup, Azure's account-verification flow required a card hold for identity verification, and no card with sufficient balance was available at the time. Rather than block the entire project on that single external constraint, the warehouse was moved to Aiven PostgreSQL — a free tier requiring no card at all.

This change does not weaken what the project demonstrates: cloud-hosted relational SQL, ETL/ELT, dimensional modeling, data-quality validation, pipeline orchestration, and scheduling are all identical in substance on PostgreSQL. The original vendor could not be used for a reason entirely outside the project's technical scope, and that reasoning is stated plainly here rather than left unexplained.

8.2 Orchestration: Azure Data Factory Dropped

Azure Data Factory was dropped from scope alongside the rest of the Azure plan. The weekly recovery/backfill pipeline that would have used it was also removed once its underlying justification (covering for a missed daily run) was judged unnecessary given the project's small batch sizes and row cap (§7.4). GitHub Actions alone now handles both the daily ingest pipeline and, implicitly, recovery — through its own retained execution history and manual rerun capability.

8.3 dim_date Declined

A conventional star-schema date dimension was considered and declined. PostgreSQL and Python can derive day, month, year, weekday, and quarter directly from failure_date at query time, so a separate, redundantly-maintained date table was judged to add storage and join overhead without a corresponding analytical benefit at this project's scale.

8.4 Fact Table Simplification

The fact table's original design included machine_failure, date_id, work_order_type, and created_at columns. All four were removed as redundant or unnecessary once the schema was implemented: every row in the fact table already represents a confirmed failure event by construction, so machine_failure and work_order_type carried no information; date_id was replaced by deriving date parts directly from the retained failure_date column (§8.3); and created_at was dropped as a load-timestamp field the project has no analytical use for.

8.5 dq_validation_log Declined

Covered in full in §7.2 — restated here as a scope decision: the dedicated data-quality log table named in the original specification was not built, in favor of pipeline-level validation plus PostgreSQL constraints.

8.6 Public Dashboard Distribution

Power BI's Publish to Web feature does not support DirectQuery reports, and the Power BI Service (which does support DirectQuery sharing) requires a work or school account that was not available. Rather than compromise on the live DirectQuery connection — which is what makes the dashboard a genuine reflection of the warehouse rather than a static export — or fall back to a DirectQuery-incompatible workaround, the report is distributed directly: the .pbix file for anyone with Power BI Desktop, plus a static PDF and PNG export for a quick look without installing anything. A dedicated read-only PostgreSQL role, `vestige_reader`, was created specifically so reviewers can open the live .pbix safely; it holds `SELECT` only, with no `INSERT`, `UPDATE`, `DELETE`, replication, or superuser privileges.

9. BI Dashboard

The dashboard is a Power BI Desktop report connected to Aiven PostgreSQL via DirectQuery — every card, chart, and slicer queries the live warehouse rather than a cached import, so the numbers below reflect whatever the daily pipeline has loaded as of the moment the report is opened.



Live Vestige dashboard, custom dark theme, screen-captured September 26, 2026

9.1 Layout

- 6 KPI cards — total maintenance events, total sites, total assets, total technicians, total failure-mode occurrences, average technicians per event
- 3 bar charts — failures by site, failures by failure mode, events by technician
- 1 line chart — maintenance events over time
- 5 slicers — failure date (range), site, technician, asset category, failure mode

9.2 Access

Distributed as described in §8.6: the live .pbix (opens against Aiven with the vestige_reader read-only credentials), a PDF export, and a PNG export, all under powerbi/ in the repository.

10. Tech Stack

Layer	Technology	Notes
Language	Python 3.13	Scripting, extraction, generation, validation, transform logic
Warehouse	PostgreSQL on Aiven (free tier)	No card required; auto-assigned region (blr / Bangalore)
DB driver	psycopg (binary) + python-dotenv	Parameterized connections and inserts
Synthetic data	Faker	Names, addresses, weighted randomization
Orchestration & scheduling	GitHub Actions	Daily ingest → validate → load pipeline
BI / Dashboard	Power BI Desktop (DirectQuery)	Live connection to Aiven, no manual refresh needed
Version control	Git & GitHub	

11. Repository Structure

The structure below reflects what is actually committed to version control. Local environment files and internal planning notes are excluded, as is standard practice — the in-depth, decision-by-decision build history they contain is instead formalized in this document.

```
vestige-data-engineering/
├── data/
│   ├── raw/
│   │   └── ai4i2020.csv
│   ├── extracted/
│   │   └── ai4i_failures.csv
│   └── docs/
│       ├── architecture_diagram.png
│       ├── schema_diagram.png
│       ├── documentation.docx
│       └── documentation.pdf
├── powerbi/
│   ├── vestige.pbix
│   ├── vestige_report.pdf
│   └── vestige_report.png
├── src/
│   ├── init.py
│   ├── constants.py
│   ├── ingestion/
│   │   ├── init.py
│   │   ├── inspect_ai4i.py
│   │   └── extract_ai4i.py
│   ├── generators/
│   │   ├── init.py
│   │   ├── ai4i.py
│   │   ├── technician.py
│   │   ├── site.py
│   │   └── ... # asset, asset_category, failure_mode
│   ├── validators/
│   │   ├── init.py
│   │   ├── ai4i.py
│   │   ├── asset.py
│   │   ├── asset_category.py
│   │   └── ... # failure_mode, site
│   └── ... # loaders, seeds, pipelines, db
├── .env.example
└── ... # requirements.txt, LICENSE, main.py, README.md
```

12. Functional Requirements Mapping

Mapped against a standard data engineer's core responsibilities.

#	Requirement	How Vestige Satisfies It
FR1	Build/test/maintain ETL/ELT pipelines	Extract → validate → load pipeline, run once as a historical backfill and daily thereafter
FR2	Write/optimize SQL for extraction, transformation, reporting	Star schema DDL, parameterized load queries, Power BI's underlying DirectQuery SQL
FR3	Design/maintain data models, schemas	Star schema (fact + 5 dimensions + 3 bridge tables), fully documented design rationale (§5, §8)
FR4	Monitor scheduled jobs, troubleshoot, root-cause	GitHub Actions run history and manual rerun support
FR5	Data quality checks, validation, reconciliation	Validation rules in §7.1, enforced at the pipeline layer plus PostgreSQL constraints
FR6	Document data flows/pipeline logic	README, this documentation, and the project's internal progress log
FR7	Collaborate, follow coding standards/version control	Git/GitHub used throughout; not otherwise demonstrable on a solo project

13. Limitations & Non-Goals

Stated explicitly, so nothing here is a limitation discovered by a reader rather than disclosed by the author.

- Not a machine-learning failure-prediction model. Vestige is a data engineering project, not an ML project — the AI4I dataset's predictive-modeling potential is intentionally left unused.
- Not built for scale. Sized deliberately small — roughly 300–500 seed events, ~20 technicians, ~10–120 sites — to stay debuggable end to end within a few days, not to demonstrate big-data handling.
- Azure/ADF not used in the final build, despite being part of the original plan, for the external reason documented in §8.1–8.2.

14. Getting Started

14.1 Setup

```
git clone <repo-url>
cd vestige-data-engineering
python -m venv .venv
.venv\Scripts\activate      # or source .venv/bin/activate on macOS/Linux
pip install -r requirements.txt
cp .env.example .env        # fill in your own Aiven admin credentials
```

14.2 Create the Schema & Backfill the Warehouse

```
python -m src.db.schemas.create
python -m src.seeds.asset_categories
python -m src.seeds.failure_modes
python -m src.seeds.assets
python -m src.seeds.sites
python -m src.seeds.technicians
python -m src.seeds.fact
```

The daily ingest pipeline runs automatically via the GitHub Actions workflow once the Aiven connection secrets are configured in the repository settings — no further manual step is required.

14.3 View the Dashboard

Open powerbi/vestige.pbix in Power BI Desktop and, when prompted for credentials, use the vestige_reader details from .env.example.